

Reporte Técnico de Práctica

Manejo de Directorios y Archivos en Ubuntu

Materia: Sistemas Operativos

Comandos: mkdir, gedit, ls -l, cp, rm

1. Introducción

El presente reporte documenta el análisis reflexivo y la justificación teórica de una práctica de laboratorio realizada en un sistema operativo Ubuntu, enfocada en la gestión básica de directorios y archivos desde la terminal. Durante la práctica se emplearon los comandos **mkdir**, **gedit**, **ls -l**, **cp** (con sus variantes **-v**, **-i** y **-r**) y **rm** (junto con **rm -r**), con el propósito de observar de manera directa cómo el sistema operativo administra el sistema de archivos: la creación de nuevas entradas de directorio, el registro de metadatos, la duplicación de información y la liberación de espacio en almacenamiento al eliminar datos.

El objetivo de este documento no es repetir el procedimiento paso a paso, sino analizar, desde una perspectiva teórica, el comportamiento que el sistema operativo manifestó al ejecutar cada comando.

2. Marco teórico

En los sistemas operativos tipo UNIX, como Ubuntu, la información se organiza en un **sistema de archivos jerárquico**, en el cual cada archivo y directorio está representado internamente por una estructura de datos llamada **inodo**. Un inodo almacena los metadatos del archivo (permisos, propietario, grupo, tamaño, fechas de acceso y modificación, y punteros a los bloques de datos en disco), pero no su nombre; el nombre se asocia al inodo mediante una entrada de directorio.

Comprender esta separación entre "nombre" e "inodo" es clave para explicar el comportamiento observado en operaciones como la copia y la eliminación de archivos, ya que estas no manipulan directamente el contenido en todos los casos, sino las referencias que el sistema operativo mantiene hacia dicho contenido.

3. Análisis reflexivo del comportamiento observado

3.1 Creación de directorios con mkdir

Al ejecutar **mkdir practica1** y **mkdir practica2**, el sistema operativo solicitó al gestor del sistema de archivos la creación de una nueva entrada de directorio dentro del directorio actual. Este proceso implica la asignación de un nuevo inodo de tipo directorio y el registro de dicha entrada en la tabla de directorios del padre (**Documentos**). Se pudo observar que esta operación es prácticamente instantánea, ya que únicamente reserva metadatos y una pequeña estructura de directorio, sin necesidad de reservar grandes bloques de datos, dado que un directorio recién creado está vacío.

3.2 Creación y edición de archivos con gedit

El comando **sudo gedit Readme.txt** abrió el editor gráfico de texto; al no existir previamente el archivo, este fue creado por el sistema operativo en el momento en que se guardó por primera vez. Este comportamiento evidencia que, a diferencia de comandos como **touch**, el archivo no se crea al

invocar el editor, sino hasta que el proceso realiza una llamada al sistema de escritura (*write*) sobre un nuevo inodo. El uso de **sudo** resultó innecesario en este contexto, ya que el usuario ya contaba con permisos de escritura sobre su propio directorio personal; sin embargo, permitió observar cómo el sistema operativo distingue entre el usuario propietario del proceso y el usuario propietario del archivo resultante.

3.3 Verificación de metadatos con ls -l

Al ejecutar **ls -l** dentro de **practica1**, el sistema operativo mostró la información almacenada en el inodo de cada archivo: tipo de archivo, permisos de lectura, escritura y ejecución para propietario, grupo y otros; número de enlaces duros; usuario y grupo propietarios; tamaño en bytes; y fecha de la última modificación. Esta salida permitió confirmar que el archivo **Readme.txt** había sido efectivamente creado y que su propietario correspondía al usuario que ejecutó **gedit**, no a **root**, a pesar del uso de **sudo** durante su creación.

3.4 Duplicación de información con cp

El comando **cp ~/Documentos/practica1/Readme.txt ~/Documentos/practica2/** generó una copia física e independiente del archivo original. A diferencia de un enlace, esta operación creó un nuevo inodo con su propio conjunto de bloques de datos, de manera que una modificación posterior sobre alguno de los dos archivos no afecta al otro. Las opciones adicionales analizadas refuerzan este comportamiento desde distintas perspectivas:

- **cp -v**: expone en pantalla cada operación de copiado realizada por el sistema, lo cual resulta útil para auditar qué archivos fueron efectivamente transferidos.
- **cp -i**: introduce una capa de protección a nivel de aplicación (no del kernel), solicitando confirmación antes de sobrescribir un inodo existente con el mismo nombre, evitando así la pérdida accidental de información.
- **cp -r**: al copiar las carpetas **vacía** e **info** de forma recursiva, el sistema operativo recorrió la estructura completa del directorio origen y replicó cada subdirectorio y archivo en el destino, demostrando que una copia recursiva es, en esencia, una secuencia de copias individuales orquestada automáticamente por el comando.

3.5 Eliminación de información con rm

Finalmente, al ejecutar **rm Readme.txt** y **rm -r info** dentro de **practica1**, el sistema operativo eliminó las entradas de directorio correspondientes y decrementó el contador de enlaces de cada inodo; al llegar dicho contador a cero, el espacio en disco quedó marcado como disponible para ser reutilizado por el sistema de archivos. Se comprobó que, a diferencia de un entorno gráfico con papelería de reciclaje, la terminal no ofrece una etapa intermedia de recuperación: la eliminación es inmediata y, en condiciones normales de uso, irreversible.

4. Justificación teórica de las observaciones

El comportamiento descrito en la sección anterior es consistente con el modelo de gestión de archivos de los sistemas UNIX/Linux, en el cual el kernel es el único responsable de mediar el acceso al sistema de archivos a través de llamadas al sistema (*system calls*) como *mkdir()*, *open()*, *write()*, *stat()* y *unlink()*. Comandos como **mkdir**, **cp** y **rm** son, en realidad, programas de espacio de usuario que traducen la instrucción del usuario en una o varias de estas llamadas al sistema.

Esto explica, por ejemplo, por qué **rm** no mueve el archivo a una papelera: la llamada al sistema subyacente (*unlink()*) únicamente elimina la referencia del nombre hacia el inodo, sin implicar un paso intermedio de almacenamiento temporal, ya que dicho comportamiento de "papelera" es una funcionalidad añadida por los entornos gráficos (como Nautilus), no por el sistema operativo a nivel de terminal.

De manera similar, el uso de **ls -l** es una aplicación directa del concepto de inodo: toda la información mostrada (permisos, propietario, tamaño, fechas) proviene de los campos del inodo asociado al archivo, y no del propio nombre del archivo, lo cual confirma la separación conceptual entre el nombre visible para el usuario y la estructura de datos interna que el sistema operativo utiliza para administrarlo.

5. Conclusiones

La práctica permitió comprobar, de manera experimental, conceptos que habitualmente se abordan de forma teórica en la materia de Sistemas Operativos, tales como la gestión de inodos, la diferencia entre el nombre de un archivo y su representación interna, y el papel del kernel como intermediario entre los comandos de usuario y el sistema de archivos.

Asimismo, se evidenció la importancia de utilizar con precaución comandos destructivos como **rm** y **rm -r**, dado que su ejecución es inmediata e irreversible desde la terminal, y se identificó en opciones como **cp -i** un mecanismo sencillo pero efectivo para mitigar este riesgo al momento de copiar archivos.

En conjunto, la práctica reforzó la comprensión del sistema de archivos jerárquico de Ubuntu y de la forma en que los comandos básicos de terminal constituyen una interfaz directa hacia las llamadas al sistema encargadas de administrar dicho sistema de archivos.